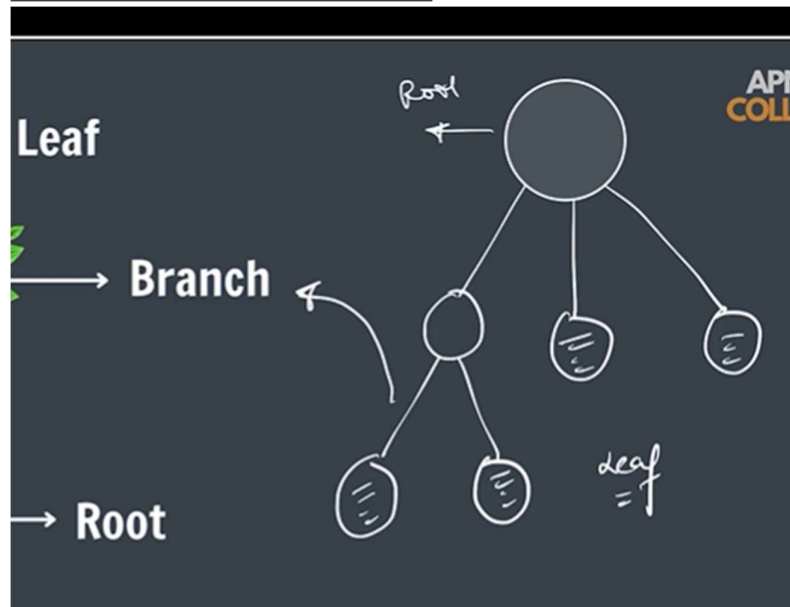
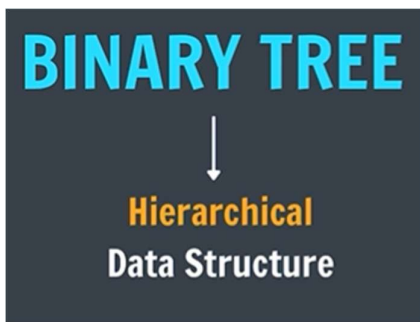
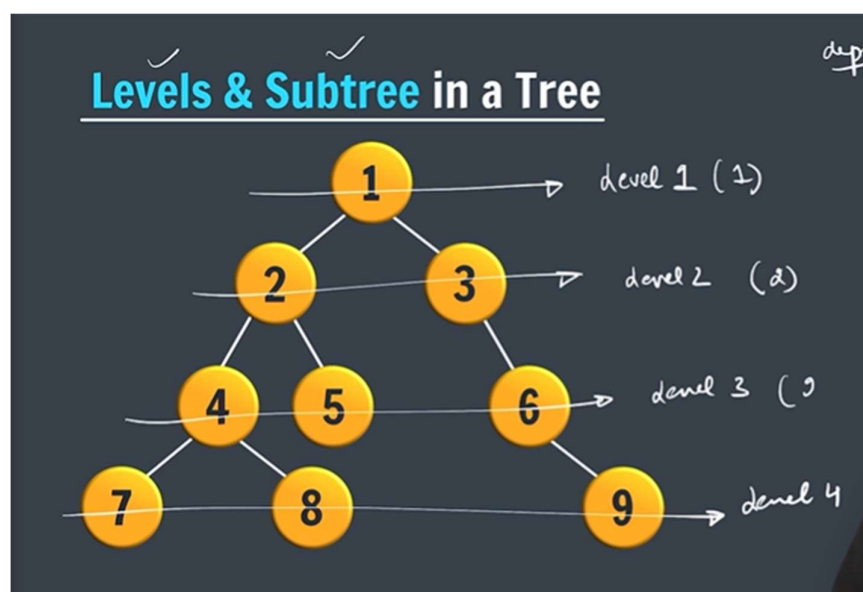


→ Binary Tree



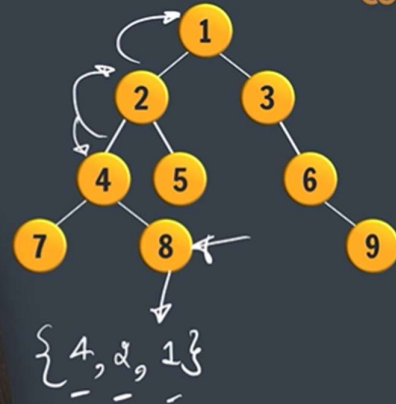
→ binary tree at most 2 children

→ Level and subtree in a Tree



## Pop Quiz

- a. Children of 4 : 7, 8
- b. No. of Leaves : 4
- c. Parent of 6 : 3
- d. Level of 2 : 2
- e. Subtrees of 1 & 2 :
- f. Ancestors of 8 :



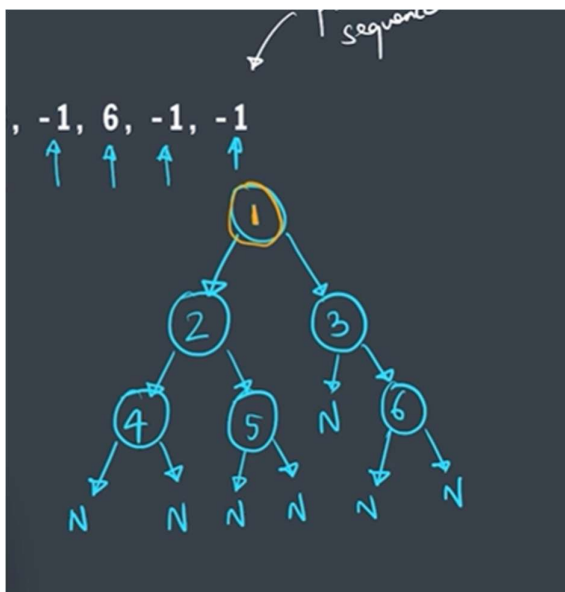
→ Build Tree Preorder

## Build Tree Preorder

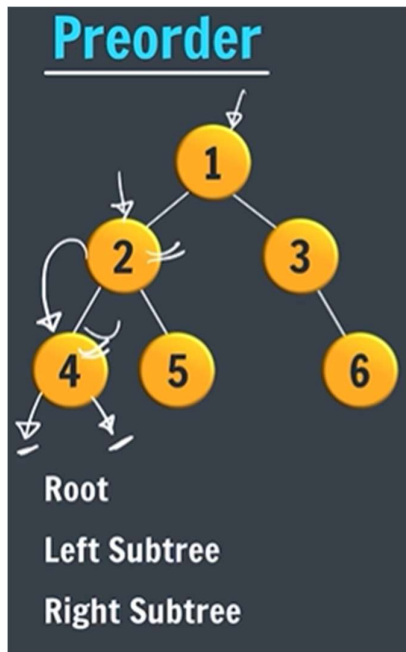
1, 2, 4, -1, -1, 5, -1, -1, 3, -1, 6, -1, -1



preorder  
sequence



→ Preorder Traversal



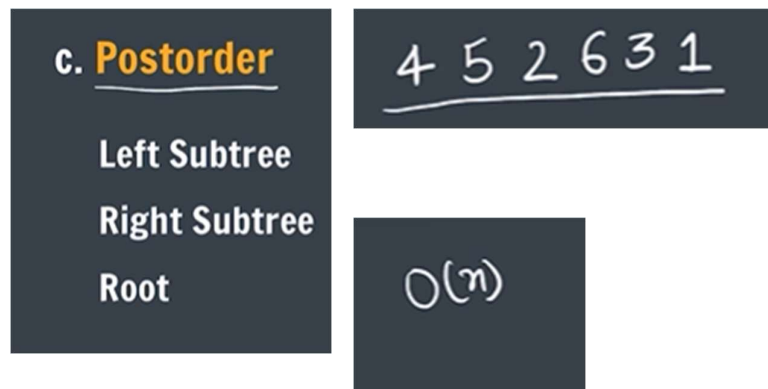
1 2 4 5 3 6

→ Inorder Traversal

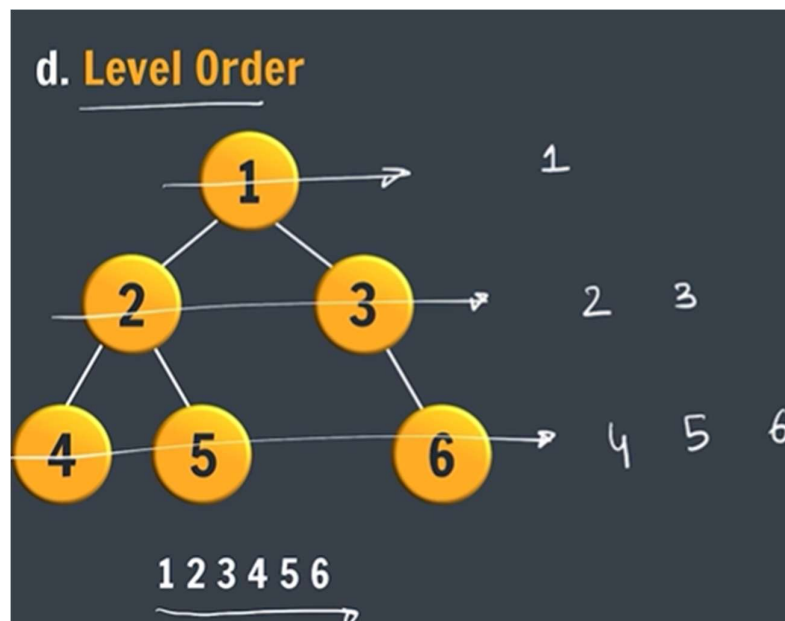


4 2 5 1 3 6 →  $O(n)$

→ Postorder Traversal

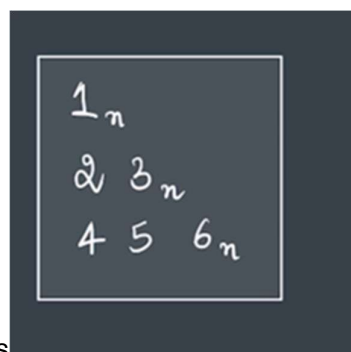


→ Level Order



-> in level order we add the one node in queue and we add the left and right nodes of that node and remove the root node

-> if we do like this we get like this 1,2,3,4,5,6

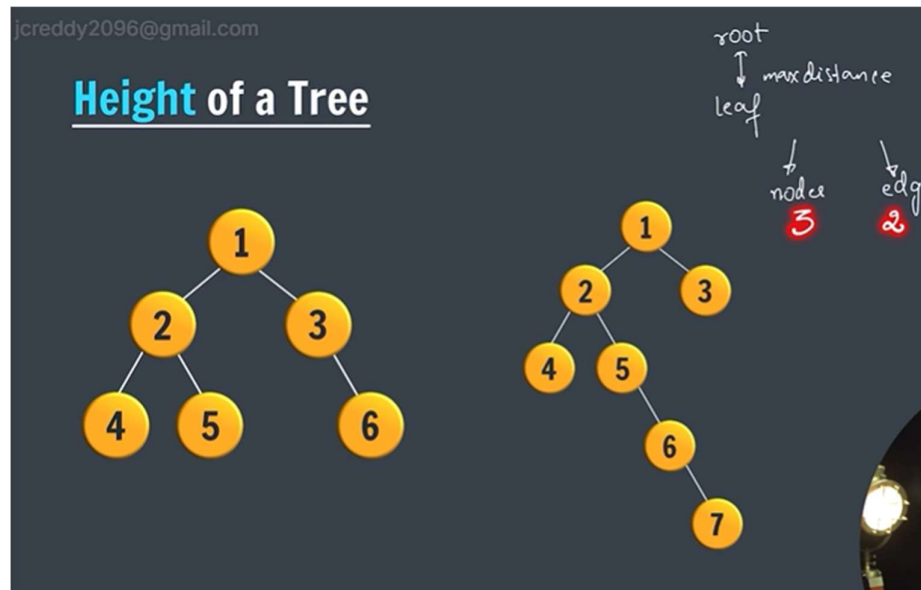


But we want like this

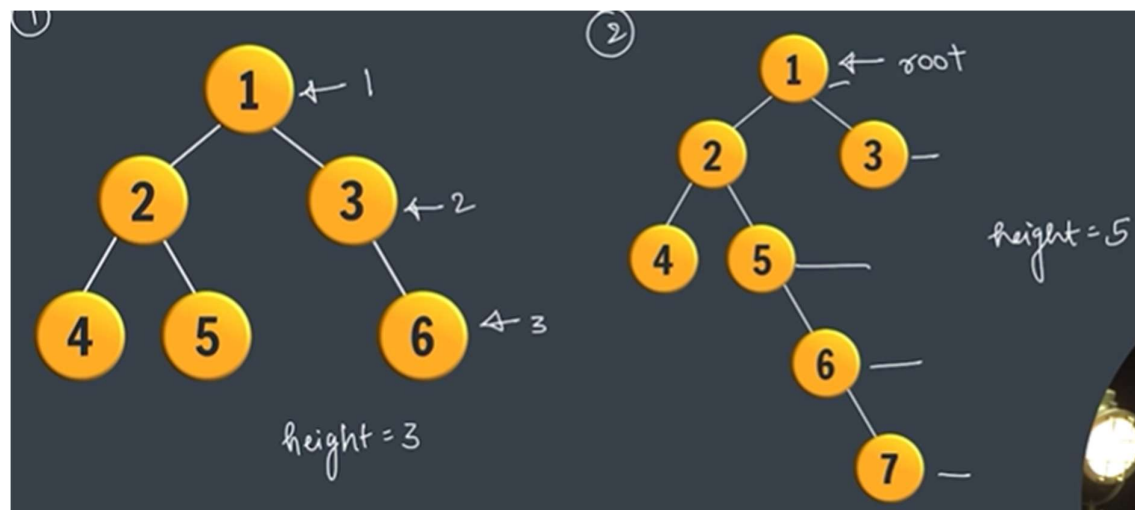
→for that we add first add root and null in the queue and after adding the left and right nodes we remove the root node and then after adding other nodes we remove the null and then again we add the null in the queue until end

→for this time complexity is  $O(n)$ .

→Height of a Tree



→we take the deepest node to calculate the height of the tree



## Height of a Tree

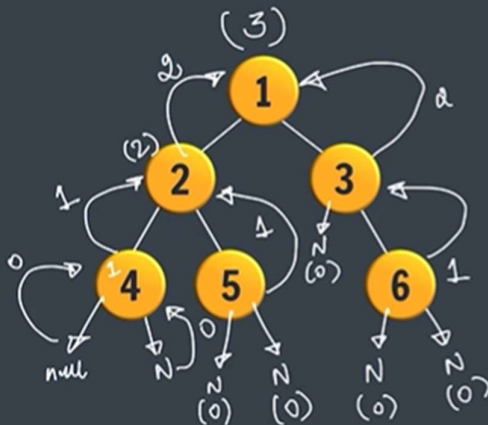


```
if (root == null)
    return 0;
```

```
lh = height(root.left)
rh = height(root.right)
height = max(lh, rh) + 1
```

## Height of a Tree

$O(n)$   
n nodes



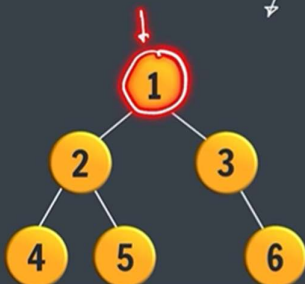
height = 3

```
if (root == null)
    return 0;
```

```
lh = height(root.left)
rh = height(root.right)
height = max(lh, rh) + 1
```

→ Count of Nodes

## Count of Nodes

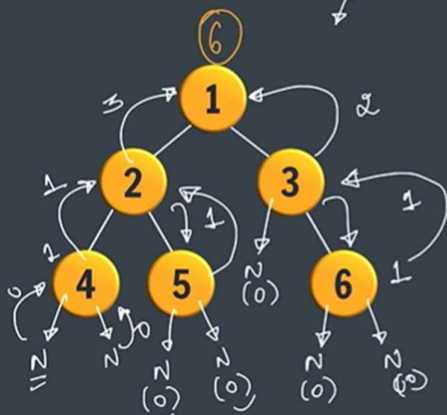


count = 6

```
if (root == null)
    return 0;
```

```
✓ lcount = count(root.left)
✓ rcount = count(root.right)
tree count = lcount + rcount + 1
```

## Count of Nodes



count = 6

if (root == null)  
return 0;

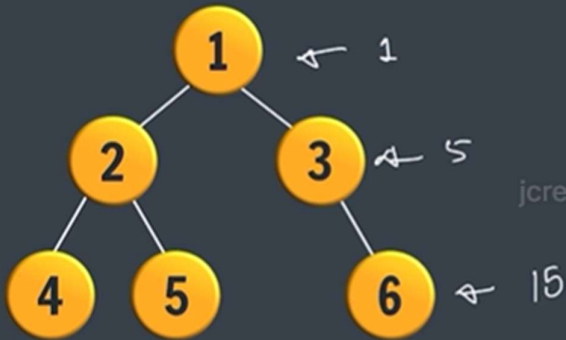
✓ lcount = count(root.left)

✓ rcount = count(root.right)

treeCount = lC + rC + 1

→ sum of nodes

## Sum of Nodes

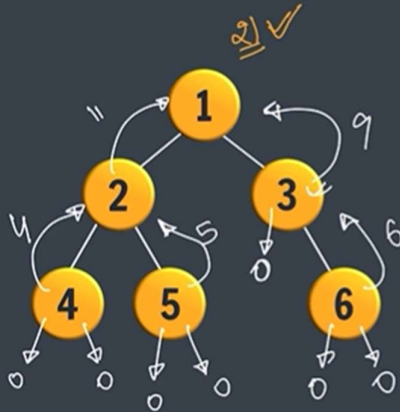


= 21

jcreddy2096@gmail

## Sum of Nodes

$O(n)$



```
if (root == null)
    return 0
```

```
leftSum = sum(root.left);
rightSum = sum(root.right);
treeSum = lS + rS + root.data;
```

→ Binary tree-2

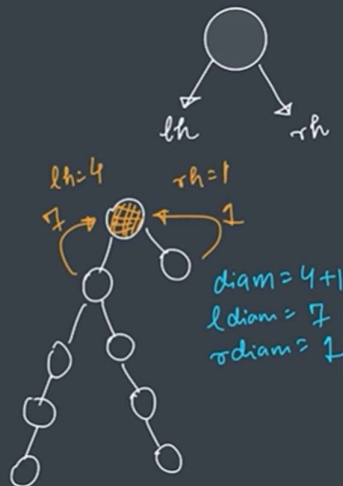
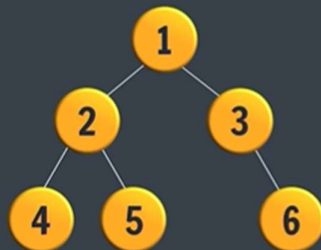
→ Diameter of a tree:

No. of nodes in the longest path between 2 leaves

## Diameter of a Tree

Approach 1

$O(n^2)$



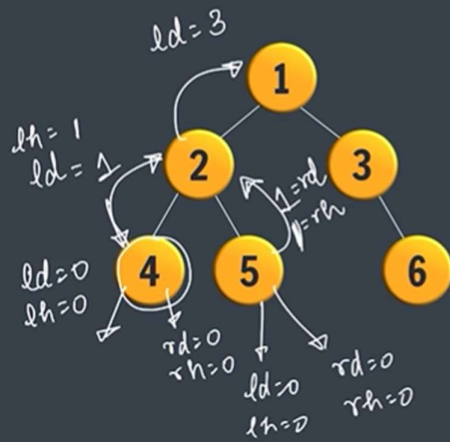
// diam passes through root  
diam = lh + rh + 1 ✓  
// diam doesn't pass  
ldiam ✓  
rdiam ✓

diam = 4 + 1 + 1 = 6  
ldiam = 7  
rdiam = 1



# Diameter of a Tree

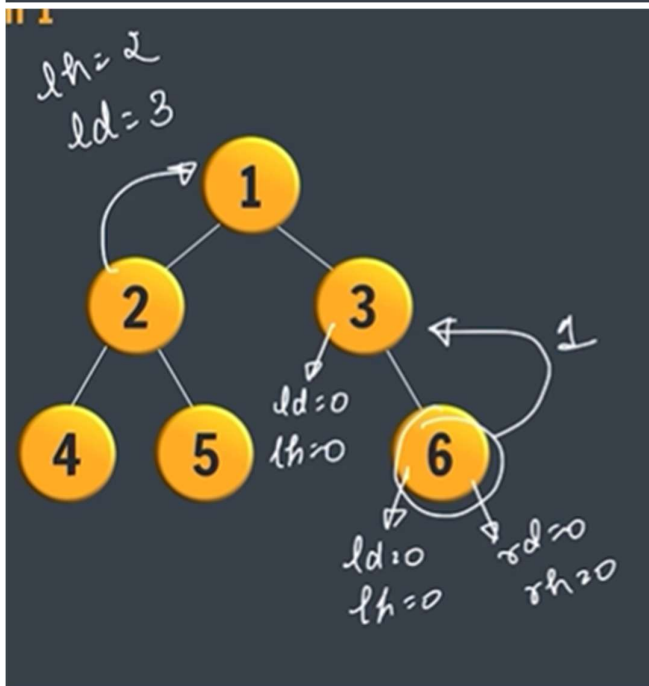
## Approach 1



$(ld, rd, self)$

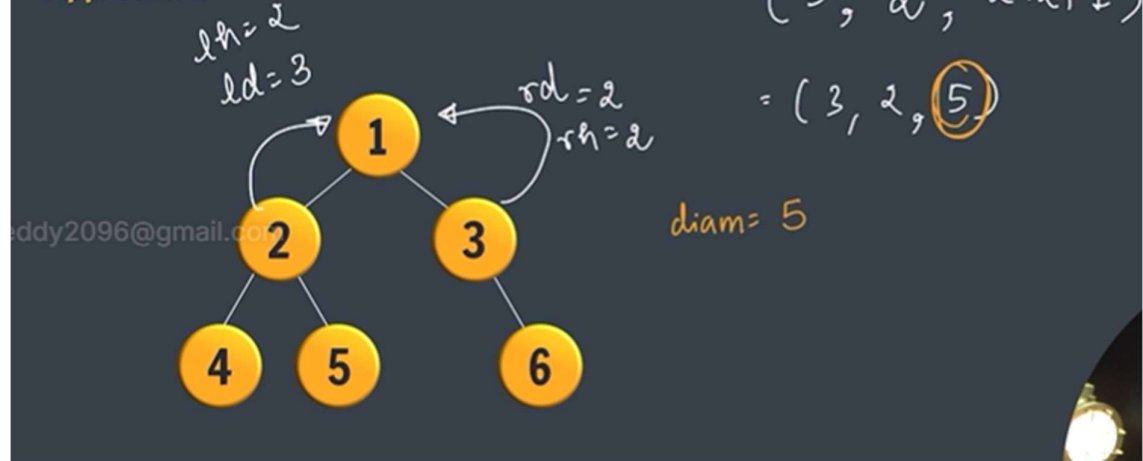
$(0, 0, 1)$

$(1, 1, 3)$



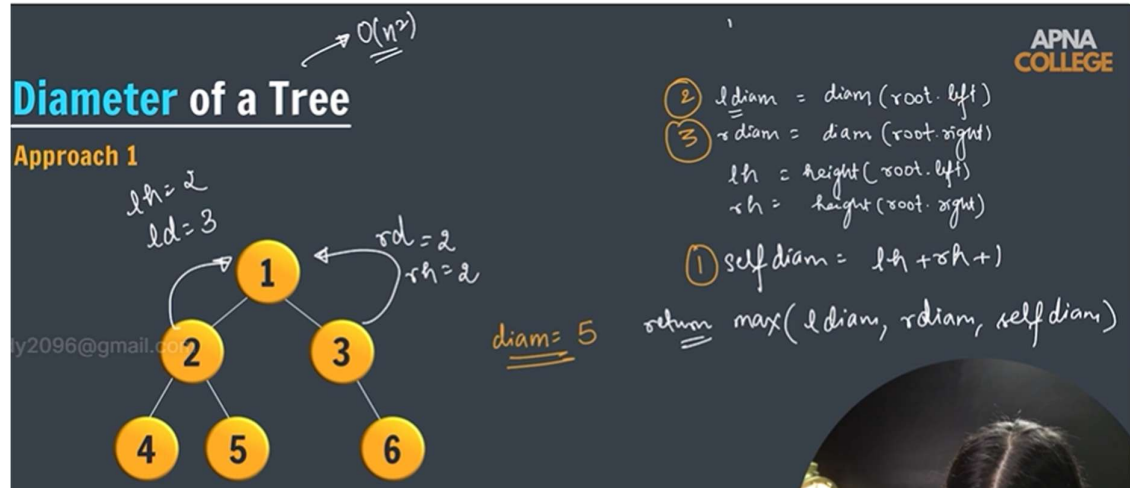
# Diameter of a Tree

## Approach 1



# Diameter of a Tree

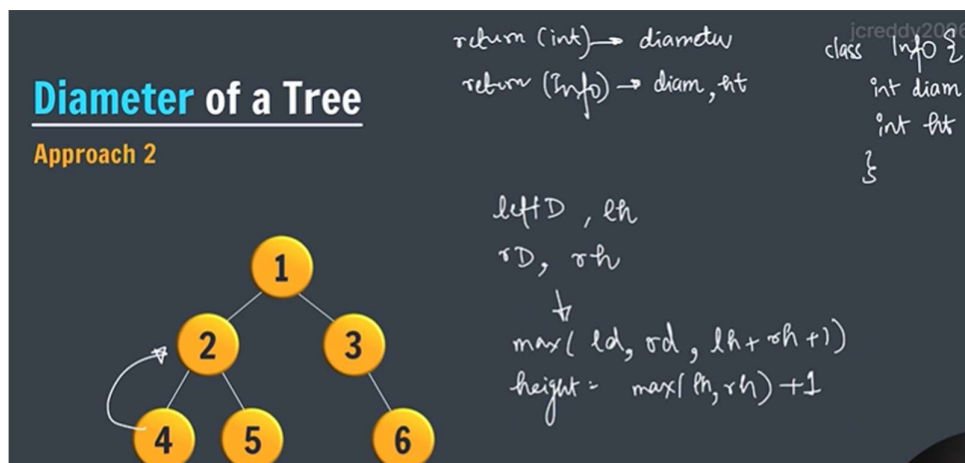
## Approach 1



→ Approach-2

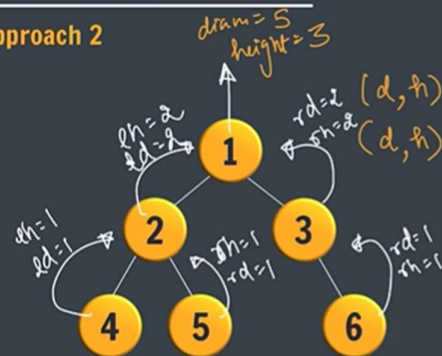
# Diameter of a Tree

## Approach 2



## Diameter of a Tree

### Approach 2



return (int) → diameter  
return (Info) → diam, ht

```
class Info {
    int diam;
    int ht;
}
```

APNA  
COLLEGE

lInfo = diam(root.left)  
rInfo = diam(root.right)

finalDiam = max(lInfo.d, rInfo.d, lh+rh+1)

finalHt = max(lh, rh) + 1

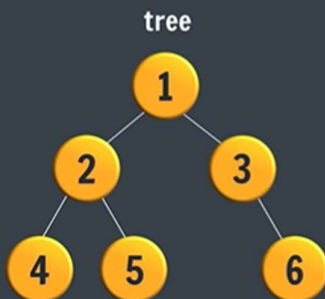
return new Info(diam, ht)

jcreddy2096@gmail.com

→ Subtree of another Tree

## Subtree of another Tree

Given the roots of two binary trees root and subRoot, return true if there is a subtree of root with the **same structure and node values** of subRoot and false otherwise.



jcreddy2096@gmail.com  
subTree

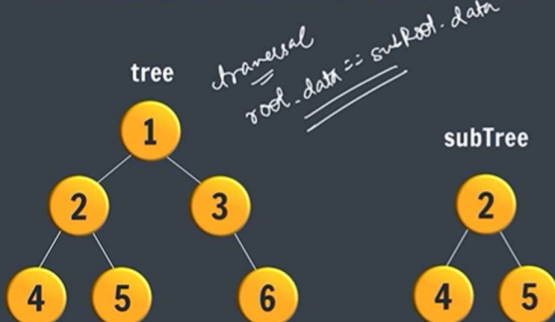


## Subtree of another Tree

Given the roots of two binary trees root and subRoot, return true if there is a subtree of root with the **same structure and node values** of subRoot and false otherwise.

- 1 find subroot in tree
- 2 check identical (subTree, node subtree)

APNA  
COLLEGE



subTree

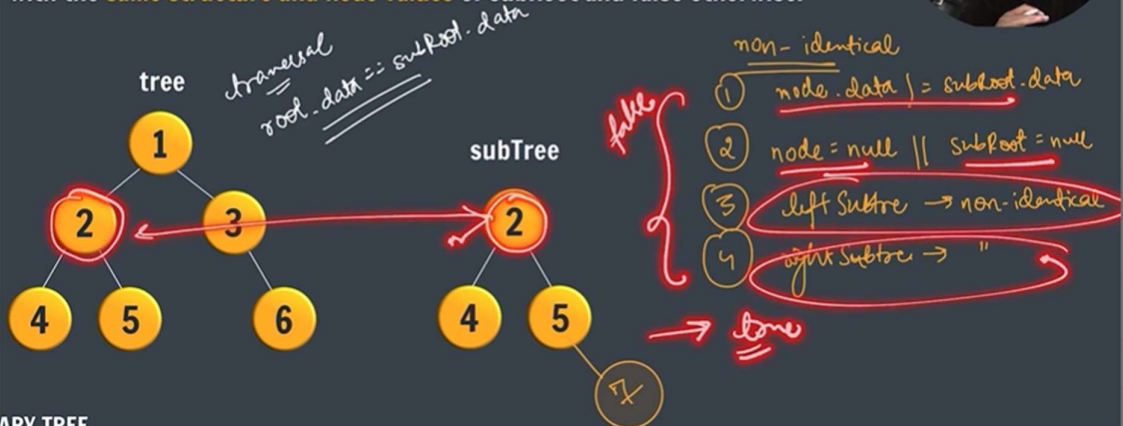


- non-identical
- 1 node.data != subroot.data
  - 2 node = null || subroot = null
  - 3 left subtree → non-identical
  - 4 ,

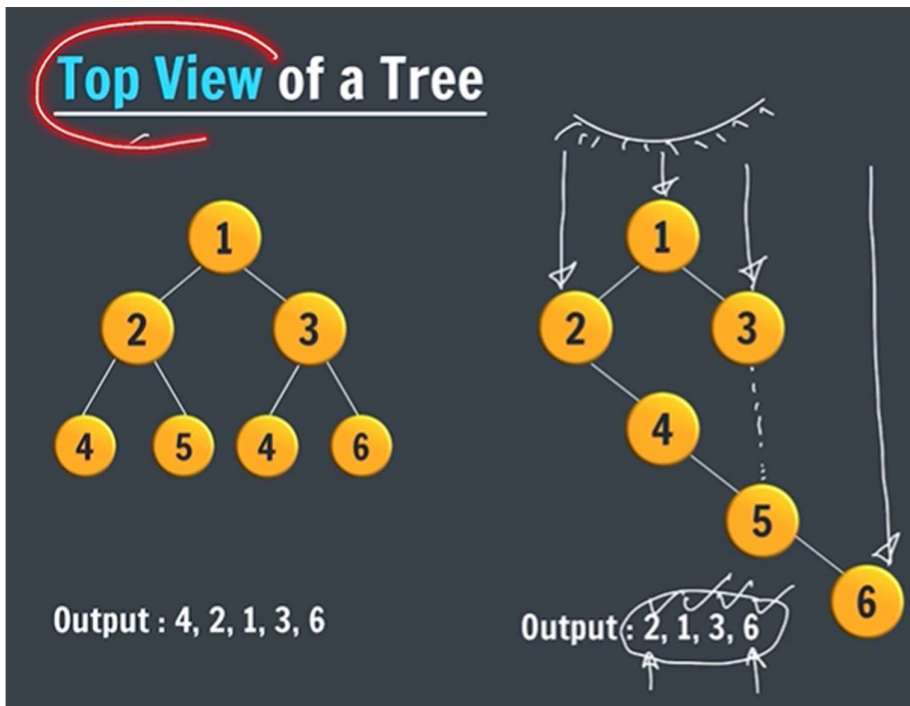


## Subtree of another Tree

Given the roots of two binary trees root and subRoot, return true if there is a subtree with the **same structure and node values** of subRoot and false otherwise.



→ Top view of a tree



→ what is map: map(key,value) key=unique value

import java.util.\*;

## Top View of a Tree

**HashMap in Java (Brief)**

Create

```
HashMap<String, Integer> map = new HashMap<>();
```

\* key = unique value

map (key, value)

| countries | population |
|-----------|------------|
| "China"   | 125        |
| "India"   | 100        |
| "US"      | 50         |

→ in hashmap add -O(1)

Remove -O(1)

Get - O(1)

import java.util.\*;

## Top View of a Tree

**HashMap in Java (Brief)**

① Create

```
HashMap<String, Integer> map = new HashMap<>();
```

Integer

② Add (put)

```
map.put (key, value)
```

③ Get

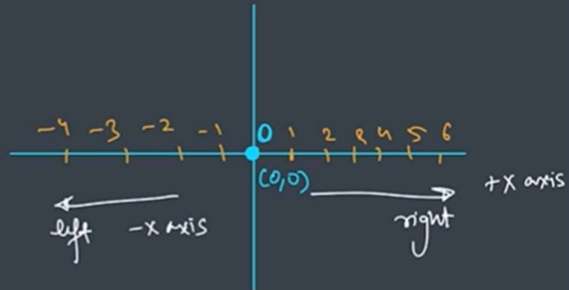
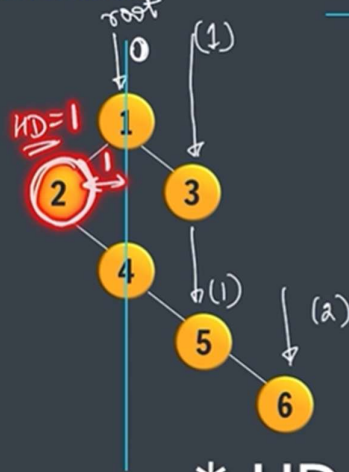
```
map.get (key)
```

jcreddy2096@gmail.com

→ in hash-map we need to enter the duplicate value like in hash-map it will replace the original value

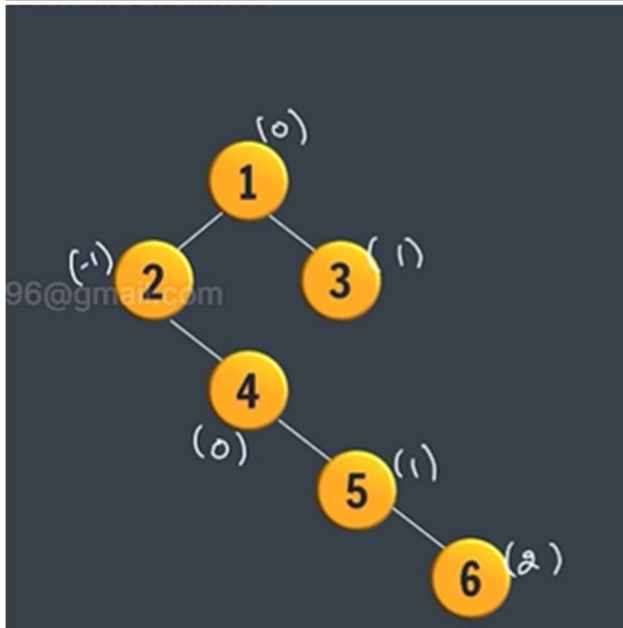
## Top View of a Tree

Horizontal Distance



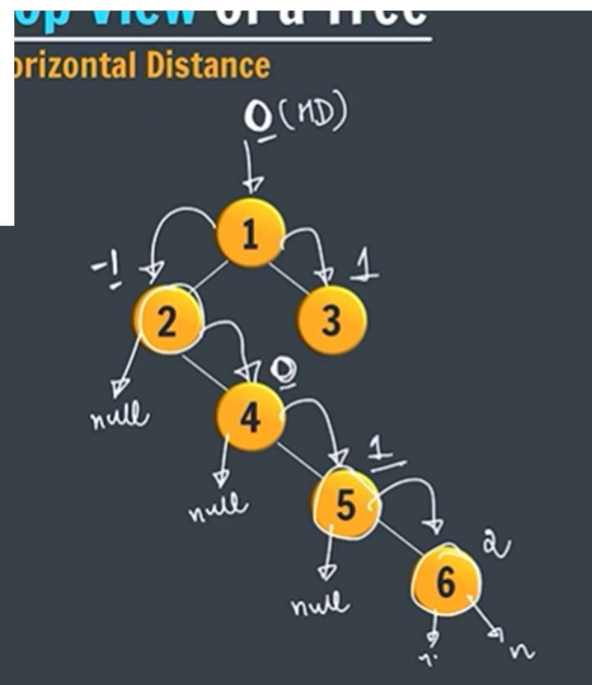
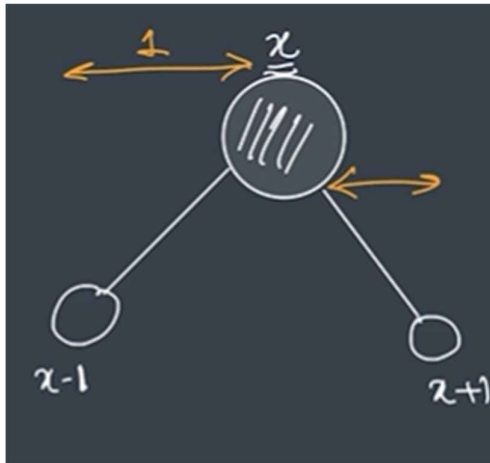
\* HD of 2 is -1

INARY TREE



| HD | node |
|----|------|
| 0  | 1    |
| -1 | (2)  |
| 1  | (3)  |
| 2  | (6)  |





→ Approach

creddy2096@gmail.com

## Top View of a Tree

**Approach**  
(Level Order Traversal)  
 $\min = 0 - 1$   
 $\max = 0 + 2$

A diagram of a binary tree with nodes labeled 1 through 6. Node 1 is the root. Node 2 is the left child of 1, and node 3 is the right child of 1. Node 4 is the left child of 2, and node 5 is the right child of 2. Node 6 is the right child of 5. Horizontal distances are indicated by arrows:  $0$  for node 1,  $-1$  for node 2,  $1$  for node 3,  $0$  for node 4,  $1$  for node 5, and  $2$  for node 6.

| HD (key) | Node (value) |
|----------|--------------|
| 0        | (1)          |
| -1       | (2)          |
| 1        | (3)          |
| 2        | (6)          |

① maps  
② ND

## Top View of a Tree

**Approach**  
(Level Order Traversal)  
 $\min = 0 - 1$   
 $\max = 0 + 2$

A diagram of a binary tree with nodes labeled 1 through 6. Node 1 is the root. Node 2 is the left child of 1, and node 3 is the right child of 1. Node 4 is the left child of 2, and node 5 is the right child of 2. Node 6 is the right child of 5. Horizontal distances are indicated by arrows:  $0$  for node 1,  $-1$  for node 2,  $1$  for node 3,  $0$  for node 4,  $1$  for node 5, and  $2$  for node 6.

| HD (key) | Node (value) |
|----------|--------------|
| 0        | (1)          |
| -1       | (2)          |
| 1        | (3)          |
| 2        | (6)          |

① maps  
② ND

loop (from min to max)

-1 → map.get(-1) 2  
0 1  
1 3  
2 6